

Using Advanced Federated Learning to Reduce Storage Cost and Latency

by

Amio Malakar Ankon
21201669
Boloy Chaki
21301205
Mashrur Shakhawat Sayan
21201221
Debashish Sarker
21201580

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Supervisor:
Dr. Jannatun Noor Mukta
Associate Professor, CSE
Director, BSDS
United International University

Thesis Group ID: T2430474
Department of Computer Science and Engineering
Brac University
February 2025.

© 2024. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

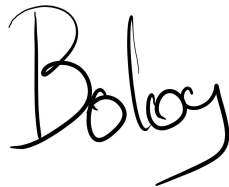
Student's Full Name & Signature:



Debashish Sarker
21201580

Amio Malakar Ankon

Amio Malakar Ankon
21201669



Boloy Chaki
21301205



Mashrur Shakhawat Sayan
21201221

Approval

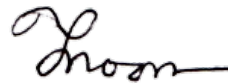
The thesis titled “Using Advanced Federated Learning to Reduce Storage Cost and Latency” submitted by

1. Amio Malakar Ankon (21201669)
2. Boloy Chaki (21301205)
3. Mashrur Shakhawat Sayan (21201221)
4. Debashish Sarker (21201580)

of [Semester], [Year] has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in [Current Semester] Year.

Examining Committee:

Supervisor:
(Member)



Dr. Jannatun Noor Mukta

Associate Professor, CSE
Director, BSDS
United International University

Co-Supervisor:
(Member)

Name of Co-Supervisor

Designation
Department
Institution

Thesis Coordinator:
(Member)

Name of Thesis Coordinator

Designation
Department
Brac University

Head of Department:
(Chair)

Name of Head of Department

Designation
Department of Computer Science and Engineering
Brac University

Abstract

In cloud computing, the use of object storage systems to store data is becoming more popular daily because of its scalability, cost-effectiveness, and distributive nature. An object storage system is a type of data storage architecture that stores unstructured data by distributing them into separate units or objects. Object storage is crucial in the cloud Internet today as cloud-based storage can sort huge data sets cost-efficiently. As the use of IaaS-based object storage systems is increasing exponentially, so is the requirement to ensure better cost efficiency and latency. Inconsistencies in data sets, reduction in performance, and storage failure are some of the most common problems in this regard. To mitigate these issues, traditional models use statistical analysis or Rule-Based systems where data is stored in a central storage. One of the best approaches to increase performance and ensure better cost efficiency could be the usage of advanced Federated Learning models such as MTFL (Multi-Task Federated Learning). Our research focuses on working and improving in this sector with reinforced learning techniques. Through reinforced learning techniques in a federated learning environment, the computer can be trained to increase performance, ensure cost efficiency, and recognize problems. Here, the approach is to use a global and local model- which works with the end devices. The end devices train the model in a local environment to store the data and ensure privacy and then send the update to the global model, which deploys the central machine learning model to all connected client devices. The state-of-the-art systems in this sector such as Amazon S3 Intelligent-tiering or Ceph use multiple access tiers or distributed storage systems to improve overall performance. We aim to achieve the same or better results using Federated Learning, which can work with a diverse range of datasets without transferring sensitive data over a decentralized data network.

Keywords:

Cloud Computing, Object Storage system, Federated Learning, Storage cost optimization, latency

Table of Contents

Declaration	i
Approval	ii
Abstract	iv
Table of Contents	v
Nomenclature	vi
1 Introduction	1
1.1 Background	1
1.2 Rational of the Study or Motivation	3
1.3 Problem Statement	3
1.4 Objective	4
1.5 Methodology in Brief	4
1.6 Scopes and Challenges	5
1.7 Key Learnings and Insights	6
2 Literature Review	8
2.1 Preliminaries	8
2.2 Review of Existing Research	9
2.3 Summary of Key Findings	14
3 Work Plan	17
4 Proposed Methodology	18
4.1 Workflow	18
4.2 Dataset Collection	18
4.3 Dataset	18
4.4 Data Pre-processing	19
4.4.1 Noise Removal	19
4.4.2 Pre-Processed Feature Representation – Comparison	19
4.4.3 Normalization – Comparison	20
4.4.4 Alignment – Comparison	20
4.4.5 Data Augmentation – Comparison	20
4.5 Training Phase	20
4.5.1 Model Selection	21
4.5.2 Training Process	22
4.5.3 Hardware	23

4.6	Evaluation	23
4.6.1	Visualization	24
4.6.2	Quantitative Metrics	24
4.6.3	Comparison	24
4.6.4	Challenges and Limitations	34
4.6.5	Result Interpretation	35
5	Conclusion	36
	Bibliography	39

Chapter 1

Introduction

1.1 Background

In today's day and age, all intelligent devices and machines are interconnected with each other through the IoT, otherwise known as the Internet of Things. All of those machines create insane amounts of data such as cookies, data packets, information, etc. These data are constantly driving the servers towards the edge of their capacity, often overwhelming and sometimes even hanging up whole complete systems. These create bottlenecks in the system that normally slow down the complete system and in severe cases may even cause the loss of important data connection and packets. This brings forth the necessity of storage optimization & latency reduction - two key aspects essential to improving responsiveness and smooth data flow in a system. In the current data-driven world, most business models rely on cloud storage, increasing the need for cost optimization. Reduction of latency, the delay in data transfer is essential for enhanced user experience. In cloud architecture, to meet these challenges, object storage systems are becoming famous day by day- which is a type of data storage architecture used to manage and store unstructured data while ensuring scalability. This considers units of data as objects that contain raw unstructured data in the forms of images, videos, etc, and metadata which is information about the object and a unique global identifier. In this paper, we are trying to optimize the storage in the object storage system and reduce the latency using Federated Learning models.

State-of-the-art models like AWS or Amazon S3 Intelligent-Tiering offer a storage class that is designed to optimize storage costs by automatically moving data by multi-storage tier storage to the most cost-effective access tier when access patterns change (What Is IaaS? - Explained Infrastructure as a Service - AWS, n.d.) [25]. Another open-source software-defined storage solution named Ceph breaks down data into objects and distributes them across the cluster to address the block, file, and object storage needs of modern enterprises (What Is Ceph? — Ubuntu, n.d.) [24]. By using Federated Learning, we aim to mitigate the risks and complexity of transferring sensitive data over multiple tiers.

According to Feng et al. [20], Federated Learning (FL) uses a decentralized Machine Learning (ML) model where multiple local models train their datasets or devices without sharing their respective datasets. The enhancement of reduced latency,

improved bandwidth usage and efficient resource allocation makes the ML training model more efficient and the use of edge computing makes it less dependent on the central server. Why choose federated learning from all the possible solutions to solve this problem? Today data security is everything. That is why federated learning was chosen because it creates boundaries between clients and servers and other servers when it comes to information sharing, according to Noor et al. [19].

In the case of object storage optimization, some work has been done utilizing Federated Learning methods. A study by Samha et al. [12] shows that in Federated Cloud Computing, CSPs collaborate to share their unused resources to improve efficiency and service quality which focuses on IaaS by leveraging virtualization, Trust Manager(TM), and Broker Manager (BM) to optimize resource allocation. Also in resource management strategy, CloudSim and cloud tool kit can be used. Asynchronous Real-Time Federated Learning (ART-FL) is being used to detect anomalies in a microservice cloud environment in a study conducted by Raeiszadeh et al. [23]. Another study by Chen et al. [15] uses asynchronous aggregation in federated learning to increase accuracy up to 65% using the FEDSAW or TEMPOW mechanism to ensure scalability, accuracy, and communication efficiency.

In this approach, the main focus is working on the constraint of storage limitation on the servers. So, to optimize the storage, all the computation and other training of the model occurs on the local devices which keeps the raw data on the local devices. There is also the bonus of having extra data security because the other devices cannot access the data processed or originated from the host devices. A study by Liu et al. [10] here is the basis of the model. The model also compresses the data when in transmission or in storage to make it more cost-efficient, which is the process of reducing the size of data by encoding it more efficiently, making it easier to store or transmit while preserving its essential information.

Some researchers have introduced other methodologies in their works on object storage optimization. A study by Kamble [16] predicts cloud resource allocation using ML, DL, and Transformer models, processing workload data like CPU, memory, and disk utilization. Another work from Durner et al. [8] showcases the usage of Any-Blob object download manager which reduces the CPU overhead and finds better latency and improves performance and efficiency. It also does not download unnecessary data to save storage space and improve cost efficiency. A Study by Noor et al. [19] uses OpenStack Swift's search functionality by integrating Elasticsearch and object detection to successfully and reliably compress and decompress huge datasets for efficient storage and retrieval.

This study tries to solve the problems plaguing object storage systems through the use of FL using data compression and latency reduction. By using FL, the model reduces computation to local devices, increasing safety and reducing data communication overhead and storage constraints. Based on existing methods like asynchronous aggregation and data compression, FL improves storage efficiency through data compression and scalability. Meanwhile integrating technologies such as Elasticsearch and object detection enhances performance for reusing the compressed data. Despite many advantages this model presents, it still has challenges in facing

system scalability and low communication overhead reliability which remains open to further research. This paper tries to lay the groundwork for efficient and secure object storage solutions in distributed systems.

1.2 Rational of the Study or Motivation

According to Griffiths [22], in the current world of IoT, it is estimated that 2 billion terabytes of data will be stored in cloud storage by 2025 and by 2029 the cloud market is projected to be worth 376.36 billion USD. Therefore, it is necessary to make cloud storage as accessible as possible. This showcases the significance of cloud storage optimization in a business model to keep up with the market and all the competition.

The main idea behind this project is to implement federated learning because the other machine learning algorithms are trained in the central server, which increases storage usage and costs. Additionally, all training data are transmitted to the central server, which increases the latency. By using Federated Learning, we can train the local models using the edge devices that reduce the load on the central server. This can result in greatly reduced storage costs and latency. Also, we can address the privacy concerns of transmitting sensitive data on the central server as we can keep the data on local devices in federated learning. Because of these reasons, we think federated learning will prove beneficial in reducing cost and latency in cloud storage, making it more accessible in the process.

1.3 Problem Statement

With the rapid growth of Cloud Computing, it has become a challenging task to optimize storage costs and reduce the amount of latency. While implementing Machine Learning algorithms in the cloud computing environment, a very large dataset is used which often leads to a high amount of data transfer that leads to high storage cost, inefficient resource utilization, high bandwidth consumption, and increased latency rate. Because of the centralized cloud server, it can create problems such as single point failure where if the central server fails the entire system will shut down. Because of a single central server, it increases network congestion because a large dataset needs to be managed. Data must travel between the central server and the user and because of this, it creates a high latency rate.

On the other hand, Federated Learning (FL) offers decentralized solutions such as improved latency where edge-based learning reduces dependency on a central server by training local models (Cloud users). Since it is a decentralized model it doesn't have to worry about single-point failure as much. However, the existing model still faces many challenges. According to Su et al. [6], FL requires frequent exchange of model updates between users and servers and this increases network load. It causes high communication costs and since many heterogeneous users in smart grids have non-independently and identically distributed (non-IID) data, their low-quality local model updates can be slower and more inaccurate than other efficient models.

Because of this, it is deteriorating the accuracy of the global model.

1.4 Objective

The primary objectives of this study are given below:

1. Enhancing System Resilience:

- To minimize bottlenecks in the system and to improve server resilience, it has to be ensured that servers do not get overwhelmed under heavy workloads.
- To maintain consistent system performance and prevent slowdowns during peak activity periods.

2. Reducing Data Transmission and Latency:

- To decrease the volume of data transmitted to servers with the use of compression, thereby reducing latency and enhancing system interactivity.
- To optimize communication processes for faster and more reliable responses.

3. Data Compression and Storage Optimization:

- To implement data compression techniques during transmission and storage to reduce resource usage and operational costs.
- To increase system reliability and efficiency while maintaining the integrity and accessibility of data.

These objectives collectively aim to create a robust, efficient, and cost-effective system capable of handling high data volumes without compromising performance or user experience.

1.5 Methodology in Brief

This study employs a federated learning framework with added data compression to reduce communication overhead while ensuring data privacy. The methodology used is as follows:

Global Model: Global central server to oversee a global machine learning model and deploy the model to all the connected client devices.

Local Model: Each end device trains its model locally and keeps the data stored in the device ensuring data privacy.

Model Update: Data is compressed before transmission to reduce communication costs. The servers receive these compressed data packets.

Aggregation on Update: The central server receives the compressed data and expands upon them for use on the global model.

Iterative Refinement: The central server broadcasts the updated global model to local devices for further training using the same compression models.

Final Deployment: The final model is deployed for real-world application and evaluation.

1.6 Scopes and Challenges

Our study aims to implement federated learning for optimizing object storage systems. This sector of research has a lot of future potential as more and more people are becoming dependent on cloud storage platforms with concerns rising over data privacy and latency. Still, the implementation also comes with some limitations. Below, we discuss the potential scopes and challenges regarding our research.

Scopes:

1. **Scalability:** The use of data compression in FL environments can make it more accessible to more devices with the use of reduced communication overhead. Thus it can be used for larger deployments like IoT or in integrated networks etc.
2. **Efficient Resource Utilization:** The use of compressed data packets in the transmission process makes it more accessible to devices that are weaker or older.
3. **Energy-Efficient Training:** Due to the small size of the data packets it will lead to overall less energy consumption in the whole system training.
4. **Adaptation to Dynamic Environments:** With the use of adaptive compression Federated Learning can adjust to dynamic situations which ensures robustness and resilience.
5. **Privacy-Preserving Communication:** With the use of methods like differential privacy can be further increased for user security and privacy with the added security on local storage.
6. **Cost Reduction:** Reducing data transfer and transfer costs is hugely beneficial to situations that are heavily resource hungry like model training and FL environments.

Challenges:

1. **Information Loss:** More compression than needed can lead to the loss of important data in the model. This can derail the performance and accuracy of the trained model which will reduce output.

2. **Balancing Compression and Accuracy:** Finding the right balance between knowing where to stop compressing for the sake of size to avoid inaccuracy is just as important. Too much compression can lead to excessive wrong training for the model.
3. **Device Heterogeneity:** Devices connected in a Federated learning environment may have different hardware and network capabilities which can make it hard for them to work together.
4. **Increased Computation Overhead:** Constant compressing and decompressing can cause older or weaker devices to slow down or freeze which can hamper the whole process.
5. **Security and Privacy Risks:** Compression can reduce the size of data packets but poorly planned methods can expose patterns making the whole model vulnerable to inversion attacks.
6. **Algorithm Complexity:** Implementation of too complex algorithms can slow down the whole model or take up too much of the resources.
7. **Compression Standardization:** Without standard compression algorithms it would be really hard for the model to work on various devices situated in various hierarchical positions.
8. **Handling Non-IID Data:** In scenarios where devices hold different, unbalanced data distributions, compression techniques may increase the odds of the biases in updates which can negatively affect the output of the model.

1.7 Key Learnings and Insights

In this study, we get valuable insight into the object storage system and various methods used to optimize it and explore how FL could be optimized in this sector. Most of the state-of-the-art cloud service providers and other researchers use multi-tiered storage and other methods where transfer of sensitive data between nodes can be risky. FL is implemented to ensure there is no risky transfer of data between the system and ensure privacy.

Apart from mitigating privacy risks, FL can improve efficient storage management, and latency reduction in cloud-based object storage systems. FL models can be used to optimize tier-based storage management and give advantages such as scalability, cost-effectiveness, and performance efficiency. Advanced FL models can distribute storage load efficiently and improve cost efficiency and scalability. Also, it reduces the dependency on the central server which helps to save bandwidth and less amount of data is transferred to the central server which helps to reduce cost storage. Furthermore, experimental results show that FL-based storage cost optimization can reduce storage costs while maintaining the highest accuracy.

From our study, our final takeaway is that FL can be a potential alternative to centralized cloud storage models because of its low latency, cost efficiency, and privacy-preserving nature. However, overcompression in FL can make the model inaccurate

and if compression methods are not designed properly, it may lead to other security issues. Also, many low-power devices are part of IoT, therefore data compression can consume additional computational resources. These issues should be addressed in future research.

Chapter 2

Literature Review

2.1 Preliminaries

This literature review mainly focuses on papers related to object storage optimization and latency reduction using federated learning. In cloud architecture, object storage systems store units of raw, unstructured data as blobs or objects. Each object has metadata containing information about the object and a unique global identifier.

In real-world scenarios, applications in business and websites need to be able to access this data fast and reliably at a cheaper cost as that is what helps them to keep up with the competition. This also brings forth the concept of data compression, where a massive amount of data is compressed and then stored or transferred among nodes. Data compression not only helps to efficiently store the data but also makes retrieving and transferring them much easier. This reduces the communication overhead delay of the data packets - which basically refers to minimizing the amount of data exchanged between systems or devices. Many works and research have thus been done in this sector, but compared to other methodologies and machine learning models, Federated learning is somewhat of an uncharted territory but has a lot of potential.

To make object storage systems even more efficient, Federated Learning can be of great advantage. It can ensure client data security while also needing less computing prowess. Some research using federated learning in this sector has introduced methods such as Asynchronous Real-Time Federated Learning by Raeiszadeh et al. [23], Hierarchical Federated Learning by Feng et al. [20], multi-model FL methods by Wei et al. [13] etc. According to Fenanir et al, [9], there have also been works using other ML models where FL is used to locally train models and ensure privacy, and the main work was done by three deep learning models- DNN, CNN, and LSTM. For optimizing storage cost and latency in a federated environment, some research has also been done on Resource Allocation using FL models- which refers to the process of distributing computing resources such as memory, storage, etc. among different users. The following literature review explains the methodologies and results of these works in detail. Most of the papers reviewed were published from 2021 to 2024, with some exceptions where we had papers from 2017 to 2020.

2.2 Review of Existing Research

The following literature review contains a detailed overview of the various methodologies tried and tested to optimize object storage systems and reduce latency which include multi-task models, various DL and ML models, RA methods, and other unique methodologies proposed by the researchers in this sector.

A study by Mukherjee et al. [11] introduces a pipeline called SCOPe (Storage Cost Optimizer with Performance Guarantees), which minimizes storage costs on cloud platforms while meeting latency and performance requirements. Here Compression Predictor (COMPREDICT) is used to predict compression ratios and decompression speeds for various data formats with high accuracy, which results in better data compression. Data partitioning is reformulated to group frequently accessed data together, minimizing read costs while optimizing storage space. Thus the process achieves cost savings of 50-83%. The pipeline is tested using the TPC-H dataset, which is a synthetic dataset. If real-world data are used, the suggested pipeline would be more reliable. Also, the process requires complex computation, which could be a bottleneck for real-time applications.

A study by Raeiszadeh et al. [23] uses Asynchronous Real-Time Federated Learning (ART-FL) to detect anomalies in a microservice cloud environment. In ART-FL, each client starts with a copy of the global model. After that, each client performs real-time learning using local data. Clients asynchronously send updated model parameters to the server without waiting for synchronization and the server incorporates them into the global model. The asynchronous updates eliminate the bottleneck of waiting for slow clients, which greatly reduces latency. After receiving the updates, the server performs feature learning to extract cross-client feature representations which helps the global model generalize better even though the updates are asynchronous. Lastly, the clients receive the updated global model for the next iteration of training. This method addresses the issues with traditional FL approaches like slow clients and enabling faster model convergence. It also scales well with large microservice systems and emphasizes real-time anomaly detection. However, it doesn't address any privacy and efficiency optimization concerns.

In a study by Feng et al. [20], the use of FL and edge computing for recommendation systems within cloud computing networks is discussed. Sharing raw data to the central server is not needed in FL which preserves privacy, while edge computing enhances efficiency by processing data in the edge devices. Delays and network stress are reduced as a result. It introduces a Hierarchical Federated Learning (HFL) framework that improves resource utilization in edge computing while addressing privacy and efficiency challenges. A decentralized Personalized Model with Neighbor Selection (DPMN) is also being used, which combines adaptive model pruning and neighbor selection to enhance model training efficiency. DPMN also addresses issues like communication overhead and the computational limits of edge devices. The process is evaluated across datasets like Fashion-MNIST, CIFAR-10, and IMAGENET but some real-world testing would make the process more reliable.

A study by Fenanir et al. [9] tries to find a method for securing IoT devices using

FL and address the privacy risks traditional intrusion detection systems have. Here FL is used to locally train models and ensure privacy. For comparative analysis, they are also using three deep learning models— DNN, CNN, and LSTM. All of these methods are evaluated on three IoT datasets: IoTID20, IoT-23, and N-BaIoT. The comparison showed that LSTM achieves the highest accuracy (99%). But the datasets used are imbalanced and there is not enough discussion on real-world deployment challenges.

In a study, Wei et al. [13] addresses the difficulties of optimizing FL for multiple models in edge cloud environments. To minimize the cost in multi-model FL, the authors created a learning optimization problem and did a joint participant selection, which in turn ensures that the convergence requirements of the FL models are met. The problem is then divided into two or three subproblems which are solved iteratively. Three algorithms are used for comparative analysis- Three Stage Greedy, Three Stage Optimization, and Two Stage Optimization. Experiments using real-world datasets like the California Housing Dataset and Diabetes Dataset from scikit-learn, FMNIST, Speech Commands Datasets, and AG NEWS Dataset show that THSO and TWSO achieve the lowest cost. The usage of real-world data gives the process reliability but the study assumes certain levels of stability in resource availability like server storage, bandwidth, etc., which always keeps changing in edge environments.

A study by Syed and Albalawi [21] tries to use machine learning to efficiently allocate resources for cloud computing services. A Kaggle dataset is used here for predictive analysis. Cost adaptation for real-world applications was also key here. They are using prior history and cloud scalability to train the model to give them a machine-learning model that would optimize cloud resource management, using predictive models, optimization algorithms, and real-time adaptation. The study has some failings as it does not have any real-world deployment. It also has unreliable data quality with challenges in scalability with inconsistent edge case handling with overreliance on the Kaggle dataset.

A paper by Chen et al. [15] tries to use a hybrid storage system for cloud-scale object storage to maintain the metadata. This hybrid model uses a mix of SSD and HDDs to access CEPH and OASIS models. Here they are using SSD for bigger data and frequently accessed data and HDD for smaller ones. This storage is also further compressed in MDS and OSD clusters. They use the RADOS service through the CLUSTER algorithm to complete this. Here cold metadata is transformed into hot metadata by metadata Migration. This model outputs lower performance efficiency when combining hot metadata and Data is disabled.

Research from Su et al. [2] builds a queuing theory-based model that retrieves data objects ensuring cost efficiency in diverse disc operations. This model uses data chunking, abstract of union operation, etc. methods to reach its target. They build the model to deal with different types of data ranging from Facebook haystack to all the way up until OpenStack cloud environments. At the frontend tier, it uses queueing latency and the distribution of request parsing latencies for the model to perform.

A study by Fang et al. [3] describes how limited network bandwidth increases the communication cost between servers and clients. The proposed method HFWP (Highly Efficient Federated Learning with Strong Privacy Preservation) is a privacy-preserving FL approach and optimizes communication overhead by reducing it. It also has an efficient optimization strategy that is divided into 3 ways. These are local computation, selective parameter sharing, and dynamic participation. These ensure data security while training. Another method is SMC (Secure Multiparty Computation) which allows multiple parties to function without exposing their data or inputs. Their study shows that, without losing performance rate, HFWP improves execution time by 20% and transmitted ciphertext size by 85% on average. It shows better accuracy than SMC and other FL methods. To improve the privacy protection of FL, HFWP uses the ElGamal encryption algorithm. It offers a lightweight encryption mechanism to protect the privacy of users without losing their accuracy. HFWP uses ElGamal encryption so it can not access individual clients' updates. Datasets like MNIST Dataset and UCI Human Activity Record (HAR) datasets are used here. Here MNIST achieved 99.75% and HAR achieved 97.5% accuracy.

A study by Khan et al. [17] describes the use of storage tiers in cloud storage optimization to reduce costs and keep up with different performance requirements. The study introduces two cost-reduction methods. One is the Rule-Based approach which is based on weights that classify as any priority score threshold and falls under four tiers of storage accesses- premium, hot, cold, archive (minimal access) and based on highest to lowest in the category of performance and highest to lowest storage cost. It also goes from lowest to highest bandwidth usage. It can work without any training data. On the other hand, Game Theory-based approaches use mathematical models of strategic models that can optimize decision-making in different scenarios in their respective storage tier optimization. The cost is calculated based on whether data remains in the same tier. Then there are proposed approaches and both are compared with each other to find optimal approaches. For the accuracy checking, Kaggle and three synthetic datasets are being used. From comparison, game theory approaches have more cost efficiency than Rule-Based approaches (33% vs 32% cost reduction) but in the case of low latency cloud storage management Rule-Based classification is more efficient.

A paper from Durner et al. [8] gives us a brief description of how cloud object storage can be used efficiently and cost-effectively by presenting a cloud object storage download manager with a brief analysis of cost, latency, throughput, and performance compared to state-of-the-art database systems. Cloud object storage faces challenges like high latency due to many concurrent requests. High CPU overhead is caused by a large set of data that needs to be analyzed simultaneously and Multicloud supports complexities, as different cloud vendors provide their own network libraries, making it difficult for users to manage multiple cloud databases efficiently. They are proposing an approach with AnyBlob and integration with Umbra DBMS. AnyBlob reduces CPU overhead and reduces latency. It uses an asynchronous I/O system to manage multiple connections in a thread. It has 3 major components which are io_uring- low-overhead system call interface, state machine-based message, and asynchronous system call. From performance evaluation, experiments were con-

ducted on AWS with different settings. They achieved maximum performance using 0.7 times the CPU resources and 1.5 times the performance under a fixed CPU budget with an additional 10% reduced CPU usage showing that AnyBlob improves the efficiency of object cloud storage by optimizing resource utilization and reducing computational overhead.

A study by Samha [12] shows how CSPs collaborate to share unused resources to improve efficiency and service quality. It focuses on IaaS and examines the application of virtualization, TM (Trust Manager) and BM (Broker Manager) are used to optimize resource allocation. TM is a component that improves trust evaluation between CU and SLA which ensures data privacy. On the other hand, BM acts between CU and CSP to ensure their cost and performance efficiency. Banker’s algorithm is a deadlock avoidance algorithm where the system never enters an unsafe state. This helps to reduce unnecessary resources that will be wasteful for FL. On the other hand, Deep Q-based is used for cloud provider ranking, improving selection strategies and cost efficiency. From their experimental result, they used simulation programs such as CloudSim, a cloud simulation toolkit to check the behavior and system of cloud components. However, the key limitation is that the study did not use real-life datasets, making it very difficult to verify the accuracy of their approaches.

A paper from Su et al. [6] presents an FL-based framework for smart grids and collaboration of Cloud-Edge to enhance efficiency and privacy in raw pirate energy data. Their proposed approach is EDO (energy data owner), which can train their local model without sharing their raw data. Still, this faces challenges like various costs during the learning process and low-quality local model updates, which prevent the global model from updating properly. To solve these issues, a two-layer DRL-based approach is implemented to make sure to discourage free-riders from using low-quality model updates. Again this system eliminates unnecessary communication overhead by prioritizing higher quality model updates. The use of Edge-Cloud collaboration reduces the burden on central data to reduce communication latency. The two-layer deep Q network (DQN) minimizes the communication overhead and improves communication efficiency and optimal pricing. Using FL on the MNIST dataset, they obtained 95.8% accuracy, which is a trade-off compared to the centralized AIoT approach’s 99.78% prediction accuracy. Also, the accuracy-sensitive ESPs ($\mu=0.8$) offer higher incentives, leading to better model performance compared to cost-sensitive ESPs ($\mu=0.4$). This will encourage EDOs to use higher-quality local models. The failings of this paper are that it only focuses security on low-quality update models and free-riding, creating limited security that does not address broader security.

A study by Kamble [16] focuses on Resource Allocation using CNN, LSTM, and Transformers deep learning algorithms that help the Cloud Service Providers (CSP) to improve cost-effectiveness, performance, and resource utilization. The authors addressed practical issues like cost, performance, and operational efficiency while leveraging real-world datasets (Google Cluster Data) for credible analysis. For evaluation of the models, evaluation parameters such as MAE (Mean absolute error), NMAE (Normalized Mean Absolute Error) & RMSE (Root Mean Square Error),

NRMSE (Normalized Root Mean Square Error), and evaluation metrics such as Hit Rate, False Alarms Rate, Miss Rate, and Resource Utilization are used. Out of the three, the Transformer model emerged as the most suitable model for this job with very high precision and effectiveness and extremely low MAE as well as a high Hit Rate. The authors stated that future research will focus on hybrid multi-objective models and prioritize security & privacy issues in this field.

Research by Ahmed et al. [14] proposes a dynamic FL method named FedOPT, which extracts data from nodes that are interconnected and have multiple layers. This FL model meets with the challenges of resource planning, and overloading in cloud architecture by using a gradient descent parameter which evaluates the resource utilization. Here, the users act as the source of resource request origin. All of the resource requests are updated to the Centralized server after being summarized. In the central server, the primary processing is done via ML techniques. The backend-based FL model assures self-learning from the patterns of data-resource allocation. To develop the FedOPT network all of the patterns are recorded and then updated to the system. The FedOPT framework executes the task of delay management and resource scheduling. Here, all of the computing is done by general ML techniques. Based on resource occupancy, The FL model increases the accuracy of prediction alongside other models. According to the authors, this resource allocation framework achieves a significant reduction in delay and can be used in multi-cluster learning environments in the future.

Research by N et al. [18] proposes an approach to solve problems related to privacy, response duration, and efficiency with a semi-supervised federated transfer learning method. The model is implemented in 2 steps - in the initial phase - by using Walrus optimization they are pre-processing and normalizing the data and selecting optimal features. After that, they train an autoencoder in every gadget in the final phase using unlabeled data which gives knowledge about the representative and less dimensional features. The centralized cloud server then combines the updates of local models trained in different nodes into the global encoder. For testing, the authors are using a dataset named SCADA, which is split into train (80%) and test (20%). Here, the performance is measured using 3 factors which are the F1 score, accuracy, and communication overhead. After evaluation, their model achieves significantly higher accuracy and F1-score compared to Non-FL models (90.41% vs 95.13% in acc & 85.99% vs 93.50% in F1-score). Whereas, compared to FL models the results were only a little bit better (95.11% acc vs 95.13% in acc & 93.21% vs 93.50% in F1-score). Overall, this study provides a framework for minimizing data requirements and lowers the amount of storage needed using Federated Learning which will play a crucial role in the future.

Research by Brum et al. [7] shows an approach to implementing federated learning in multi-cloud scenarios to improve the costs and execution time. The authors address the issue of messages sent between client and data achieving a very large size which takes more time to be transferred in cloud-based services and are experimenting with a multi-cloud scenario. For this, they used Google Cloud Provider and Amazon Web Services. To conduct the experiment, the authors execute the proposed FL approach in a classification problem for Tumor-Infiltrating Lympho-

cytes (TIL). In the training phase, each WSI is divided into smaller patches by the application for them to be classified into positive and negative. These are then used in CNN as the training dataset. The CNN classifies these patches into positive and negative while the TIL maps are created by the application in the production phase. This centralized approach uses two separate datasets for training and testing where the authors divide the datasets among all clients in a homogenous manner. While evaluating final results, the cost of service is also factored in, where GCP presents better results compared to AWS with comparatively less synchronization time, computing time, and message exchange cost.

2.3 Summary of Key Findings

The studies reviewed in this paper show that in most cases, federated learning models are being implemented to ensure data privacy. With the current situation of IoT, most CSP (Cloud Service Providers) like AWS, Ceph, and GCP use multi-storage classes or offer services that use the integration of ML/DL models. But in these cases the risk of raw data transferring remains, this is where many of the authors have used FL models as a pioneering solution. Models like HFWP, and FTL address the privacy and security challenges which also deal with response duration. Some scholars have worked on multi-storage tiering for storage optimization and latency but in these cases, there is a scarcity of real-world datasets and ML models generally use synthetic data and assume a certain level of stability that is not always the case in cloud storage services. Another pattern of research that overlaps with this sector is Resource Allocation methodologies using FL-based frameworks which is crucial in terms of latency and performance in the broader narrative of cloud environment. To directly improve on the performance issues, edge computing is used to address the limitations of traditional approaches. Some unique methodologies are also seen such as the queuing theory-based model for cost reduction but these are not future-proof as most of the research work done in this sector has heavily emphasized and stated the importance of multi-task or hybrid models in the market. The most promising works in this regard use Federated Learning with other ML techniques where the general computation process is done by the traditional ML models where FL adds on a value with enhanced privacy and prediction based on resource occupancy. These hybrid and multi-task models can work with both resource overloading and resource planning as well as other optimization factors such as storage cost and latency.

Author name	Year Published	Used algorithms, architecture	Dataset	Sector of optimization	Use of federated learning?
Raeiszadeh et al. [23]	2025	ART-FL, GNN, PU, SCG	TrainTicket	Anomaly detection	Yes

Author name	Year Published	Used algorithms, architecture	Dataset	Sector of optimization	Use of federated learning?
Qi et al. [20]	2024	HFL, PFL, FEL	FMNIST, CIFAR-10, and IMA-GENET	Communication efficiency, privacy and resource optimization	Yes
Fenanir & Semchedine [9]	2023	SID, DNN, CNN, LSTM, IDS	IoTID20, IoT-23, N-BaIoT	Information security and user Privacy	Yes
Mukherjee et al. [11]	2023	SCOPEe, Multi-tier cloud storage, DATA-PART	TPC-H	Storage & access cost	No
Wei et al. [13]	2023	HFL, DFL	FMNIST, Speech Commands Datasets and AG_NEWS Dataset	Training cost optimization	Yes
Kamble et al. [16]	2024	LSTM, CNN, BERT	Google Cluster Data	Resource utilization, cost-effectiveness and performance	No
Ahmed et al. [14]	2024	FedOPT, GNN	N/A	Resource Utilization	Yes
N et al. [18]	2024	IDS, GRUs, FTL, FNN, SVM	SCADA	Privacy and security	Yes
Brum et al. [7]	2022	Cross Device-FL, CNN, TIL	TCGA-TIL Dataset	Execution time & cost of FL application	Yes
Shinde et al. [5]	2021	HAP, RSU, VN, VEC	N/A	Latency & computation offloading	Yes

Author name	Year Published	Used algorithms, architecture	Dataset	Sector of optimization	Use of federated learning?
Fang et al. [3]	2020	DP, HFWP, ElGamal encryption	MNIST, HAR Dataset	Privacy preservation	Yes
Khan et al. [17]	2024	Thompson Sampling algorithm, Greedy algorithm	Synthetic and Semi-synthetic dataset	Storage cost optimization	No
Durner et al. [8]	2023	AnyBlob, Object Scheduler, Object storage architecture	N/A	Storage cost optimization and high performance data retrieval	No
Su et al. [2]	2021	Arima, LSTM, DQN, PSO, Linear regression	DevOps, AWS, and Azure Effectiveness Deployment Dataset	Private Data Sharing in edge-cloud collaboration	Yes
Samha et al. [12]	2023	Deep Q, Banker's algorithm	N/A	IaaS cloud design and resource management	No
Syed & Albalawi et al. [21]	2024	DQN, K-means, ARIMA, LSTM	Kaggle dataset	Optimization of resource management	No
Chen et al. [15]	2024	XCeph, COSBench	COSBench Workload	Metadata maintenance	No
Su et al. [6]	2017	Consistent Hashing, Load balancing	N/A	Response latency requirement	No

Table 2.1: Table of Key Findings

Chapter 3

Work Plan

The following figure shows the proposed work plan for the entire research. There will be two synthetic datasets provided for our testing, both datasets will be split into two parts- 70% for training and 30% for testing. For both datasets, there will be these general attributes- Object ID, Object Size, Object Type, Timestamp, Storage Class, Replication Factor, Client ID, Access Frequency, Data Compression Level, Network Bandwidth, Storage Medium. For the latency dataset there will be these unique attributes- Read/Write Latency, Proximity to User, Transfer Bandwidth, Queue Depth, Server Load, and Access Pattern and for the storage optimization dataset there will be- Data Retention Period, Storage Location, Storage Pricing Model, Total Dataset Size, Object Popularity. These datasets are provided to the end devices, which will be compressed first to save time on transmission. There, the local model will revert these packets and train the local model which will then send this to the global model. Afterwards if accepted the global model will send this to all local models. All communication will be done here efficiently to save cost. Afterward, the compressed data will be stored for further use in the future.

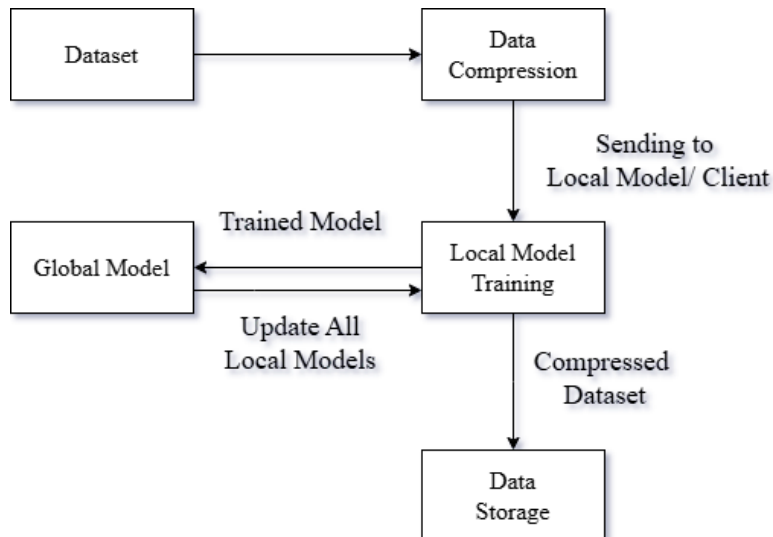


Figure 3.1: Work Plan

Chapter 4

Proposed Methodology

4.1 Workflow

Three Federated Learning algorithms- FedAvg, FedOpt, and Fedprox were used in HAR90, CIFAR10, and MNIST datasets, where lightweight CNNs with two convolutional layers were used. For HAR90 and CIFAR-10, simulations were done using Pytorch framework, and Tensorflow was used for MNIST. All of the client and server communications were simulated in a single machine using CUDA. After the federated training rounds, the model evaluation was performed in the centralized test dataset.

4.2 Dataset Collection

The HAR90 or Human Activity Recognition- 90 dataset was collected from UCI datasets, which consisted of raw inertial sensor data, which were data of physical activities collected from smartphone sensors. These were all sorted into 6 human activity classes.

The CIFAR-10 dataset was collected from the torchvision datasets, which contains real-world images of airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck - sorted into 10 image classes.

The MNIST dataset was collected from Keras datasets, which had black and white images and was divided into 10 image classes from 0 to 9, and the pixel size of each image was 28x28.

4.3 Dataset

The HAR90 dataset contained a total of 48392 windowed sequences after preprocessing, where each of the sequences had 1600 features from the gyroscope data. These sequences were divided into * activity classes, where each class had 6049 samples.

The CIFAR-10 dataset had 60000 total samples, where 50000 were used for training and 1000 were used for testing. It was divided into 10 balanced classes, with each

containing 6000 images. Each image had 3 channels (RGB).

The MNIST dataset contained a total of 70000 grayscale images, which were distributed into 10 balanced classes and reshaped and normalized for CNN input format (28,28,1).

4.4 Data Pre-processing

Techniques used to handle missing values, outliers, and inconsistencies.

4.4.1 Noise Removal

The approach to noise removal was really diversified. A lot of it was based on the datasets and their characteristics. In HAR90, the data in question was from smartphones. So this dataset had a lot of clutter and noise induced due to user movements. To mitigate these problems, the raw data from sensors was segmented carefully, and fixed-length windowing was used on them. It was quite a different case for CIFAR-10. The images from this dataset were quite clean and did not require any denoising. But some implicit noise mitigation had to be done through random cropping, horizontal flipping, etc. Compared to them, MNIST was really simple. The grayscale images were already pretty clean, and any variation from the training path was handled by the CNN to fix minor imperfections. Here, the noise reduction was highly dependent on the data modality, as time-series datasets like HAR90 required demanding structural corrections, while MNIST and CIFAR-10, the image-based datasets, were free of those due to model robustness and light augmentation.

4.4.2 Pre-Processed Feature Representation – Comparison

Transformation was applied to all the datasets, but the specific method varied depending on the dataset. This difference was based on their characteristics. These datasets were transformed based on the nature of their data in each case. HAR90 used a custom pipeline to flatten the segmented time-series windows into 1D vectors, where they were processed into a 1D CNN. A custom script in TensorFlow handled the preprocessing, converting the segmented data into 1D tensors with the appropriate structure for model input. CIFAR-10 was more straightforward, where the raw pictures only needed to be transformed into 3D RGB format with the use of standard PyTorch transformations. MNIST was grayscale 2D images, which were turned into 3D tensors using a single channel dimension. After preprocessing, all datasets were converted into tensor formats compatible with CNNs, though they still differed in structure and dimensionality. These differences directly influenced the model designs. Like in HAR90, the model started with MLP and later shifted to 1D CNNs, whereas CIFAR-10 and MNIST were pre-processed using only lightweight 2D CNN architectures.

4.4.3 Normalization – Comparison

Normalization was used across all the datasets, but the methods and techniques used varied depending on their characteristics and structure. In HAR90, normalization meant standard feature scaling, which smoothed the convergence and brought different sensor axes to a closer, comparable range. On CIFAR-10, the method applied was per-channel normalization, where the dataset determined the mean and the standard deviation values. This, in turn, was helpful for deeper CNNs and color image data. Lastly, on MNIST, the dataset was scaled down from 0–255 to 0–1, which helped simplify the dataset. Overall, normalization stabilized the training of these datasets and improved the model convergence. But each dataset needed a different approach based on its structure and statistical profile.

4.4.4 Alignment – Comparison

The most important task in this whole process was the alignment of the input samples from the datasets, even though different paths and mechanisms were used to reach this destination. Out of the three, HAR90 was the one that needed manual handling to make sure each segmented section was at its correct activity label. Due to the temporal and sequential nature of this dataset, the proper alignment of the data was of utmost importance in supervised learning. CIFAR-10 and MNIST used built-in dataset loaders from PyTorch and TensorFlow, respectively, that did the work of label matching. Extra care was given during the splitting to make sure federated client subsets got the proper representative class distributions. The other two visual datasets relied on the framework more extensively, whereas HAR90 needed extra attention to preserve semantic integrity throughout preprocessing.

4.4.5 Data Augmentation – Comparison

In data augmentation, practices diverged based on the demands and characteristics of each of the datasets. Unlike before, here, CIFAR-10 needed more care in augmenting the dataset. Augmentation was used as a regularization technique here, with random cropping and flipping to improve the model’s generalization and diversity within small image distortions. HAR90 and MNIST did not need synthetic augmentation. Here, diversity was achieved by distributing the data across simulated clients to reflect realistic heterogeneity in federated settings. In such cases, the variation in local data distribution was used as a proxy instead of augmentation. This creates a sufficient challenge for the model, which had to generalize these differing client data. This contrast highlights the flexibility of federated learning setups in incorporating structural variation without relying on traditional image-level augmentation techniques.

4.5 Training Phase

The training phase is critical for evaluating model performance in a federated learning environment. It allows each simulated client to train locally on its data, without sharing raw data with the global model. This process, repeated over multiple rounds

using algorithms. It is essential for measuring accuracy, latency, and memory usage across different models and datasets.

4.5.1 Model Selection

To start our training process, we first selected the Multilayer Perceptron model (MLP). It is a type of Artificial Neural Network (ANN) used in machine learning, which is a commonly used type of feedforward neural network. However, for its poor performance, we later switched to a 1-Dimensional Convolutional Neural Network (1D-CNN) is a deep learning architecture that performs convolutions along one dimension. We also selected a CNN model with 3 convolutional layers and 2 fully connected layers, and a lightweight CNN model was used with 2 convolution layers with 2 dense layers. These models are used to find out the latency, memory overhead in federated learning algorithms like FedAvg, FedOpt, and FedProx in various datasets.

Multi-Layer Perceptron (MLP):

According to Mia et al. [1], an MLP consists of more than one or more hidden layers with non-linear activation functions, and an output layer is suited to the target task. During training, it uses the backpropagation algorithm to minimize the prediction error. Through the network, it takes input, and the error between predicted and actual outputs is calculated using a loss function like cross-entropy.

Here is an image of how backpropagation of an MLP works-

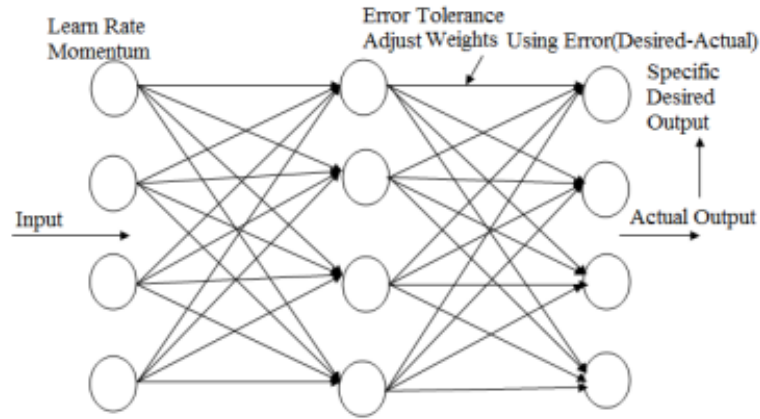


Figure 4.1: Multi-Layer Perceptron

Convolutional Neural Network (CNN):

This model is well-suited for a type of deep learning model for analyzing visual imagery. We used a one-dimensional CNN (1D-CNN) for training datasets like HAR90. We utilized this to process the feature sequence extract. According to Kiranyaz et al. [4], the input signals are passed through multiple layers of this model, where each layer applies a learnable filter to detect patterns in data. Batch normalization for stability, ReLU activations for non-linearity, and pooling layers for dimensionality reduction and translational invariance enhancement come next. In order to

avoid overfitting, dropout is sometimes used during training. Also followed by two other CNN variants, which use more convolutional layers to increase their depth and capacity to learn complex features. These enable deeper hierarchical feature extraction from sequential data. Deeper convolutional layers allow the network to acquire more abstract and discriminative characteristics over time, resulting in improved temporal pattern identification. Finally, the output layer uses a softmax activation function to produce a probability distribution over target classes.

Here is an image of a 1D-CNN model:

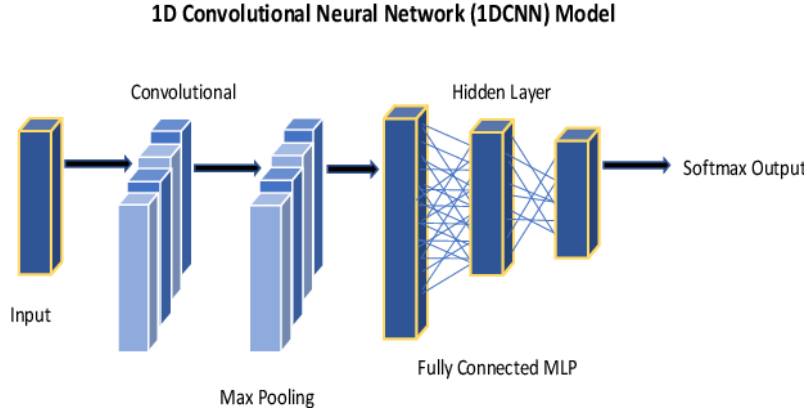


Figure 4.2: 1D-Convolutional Neural Network

4.5.2 Training Process

The training phase is a critical phase in developing and evaluating our models and algorithms within federated learning frameworks. Training the models allows us to recognize these training datasets. In our setup, the training process simulates many client trains and models on its own local data without sharing actual raw data. Once the training is done, each client sends its model update to a simulated central server, where it combines these updates and improves the global model. For each dataset, our method is different.

MNIST dataset-

To evaluate the performance of federated learning algorithms on the MNIST dataset, first, we use separate training scripts. For each file, it works on a different algorithm like FedAvg, FedOpt, and FedProx. In each file, we use the same number of samples in the training dataset, or it was already organized which samples to train and test. Here total of 60,000 samples were used in the training dataset, and 10,000 samples were used for the test dataset. For the client side, we simulated over 100 clients and distributed clients with an equal amount of the training dataset. An MLP model was used for this, but the result was not good enough. So later lightweight CNN model with 2 convolution layers with 2 dense layers was used for evaluation. These simulated clients independently trained the model on their local data using mini-batch Stochastic Gradient Descent (SGD). After each training round, the clients sent their models update to the central server. The global model updates the server

by aggregating these updates for each algorithm. From round one, we get less accuracy, and gradually increase the round numbers, we start to get higher accuracy rates. Also, with a different round model, training time gets different times and memory overhead.

HAR90 dataset-

After the pre-processing phase for this dataset, we split the dataset for training and testing. Same way as the MNIST dataset, we made several scripts for each algorithm. Here MLP model was also used during startup, but with very poor performance, we had to switch to the 1D-CNN model. The accuracy rate was much higher than the MLP model. For the client side, we simulated 100 clients and distributed clients with an equal training sample. Same way model trained and send update to the server. To generate a perfect training model, we had to simulate 10 rounds. But with the FedOpt algorithm to get perfect accuracy, latency, and memory overhead, we had to simulate a total of 70 rounds.

CIFAR-10 dataset-

Like the MNIST dataset, CIFAR-10 is a labeled image dataset that contains color images, and it is commonly employed for training and evaluating machine learning models. It contains 60,000 samples with 50,000 training images and 10,000 test images. Here, we also used 100 simulated clients. Local training on each client was performed using mini-batch Stochastic Gradient Descent (SGD) to optimize the model parameters. Here, we used a lightweight CNN model with 3 convolutional layers and 2 fully connected layers. Here, we also separated the scripts file for different algorithms. From each algorithm, we get different accuracy rates with different latency and memory overhead. To get an optimal result, it took over 30 Rounds to get a perfect result.

4.5.3 Hardware

Hardware plays an important role in a reliable and efficient federated learning system. Although federated learning typically involves training across multiple devices, for simulation, we had to use a single computer. A CUDA-enabled GPU was used to accelerate both client and server-side model aggregation. To monitor system performance and resource usage, the psutil library was used throughout the entire training process. The hardware components we used were an NVIDIA RTX 4070 (8GB V-RAM) GPU, 64 GB of RAM, Intel Core i5 CPU. These are efficient enough for intense computational work.

4.6 Evaluation

In the evaluation phase, we assess the performance, efficiency and resource usage of the federated learning models to determine their effectiveness for optimizing storage cost and reducing latency. The evaluation metrics include accuracy, confusion matrix, precision, recall, F1 score, loss, correlation matrix, latency, memory usage, storage cost overhead, communication overhead and convergence rate.

4.6.1 Visualization

To visualize the metrics, we plotted accuracy, loss, precision, recall, F1 score, model size, RAM usage and communication overhead per round for each algorithm and datasets. At the final round for each algorithm, we visualized the confusion matrix for class-level performance. Also, we visualized the correlation matrix to measure the relation between true and predicted labels.

4.6.2 Quantitative Metrics

We are using accuracy, precision, recall, F1 score and confusion matrix to evaluate the models' ability to predict the labels. To visualize how efficiently the model learns, we are calculating loss and convergence rate. Also we are showing correlation matrix to visualize the prediction patterns by highlighting relationships between predicted classes. Finally we are trying to find practical feasibility by comparing latency, memory usage, storage cost overhead and communication overhead.

4.6.3 Comparison

For HAR90 dataset, the highest final accuracy was achieved by FedAvg, which is around 99.84%. FedProx also achieved near peak accuracy which is around 99.38%, though it showed slightly more fluctuations early on compared to FedAvg. FedOpt showed the lowest accuracy which is around 97%, but it had faster convergence. Both FedAvg and FedProx had the model size of around 0.6 MB, but FedOpt had a much larger model size of 12 MB. We found that RAM usage was similar across all three algorithms. FedOpt had the lowest latency per round, followed by FedAvg and FedProx. Among the three algorithms, because of the larger model transfers, FedOpt had the highest communication overhead.

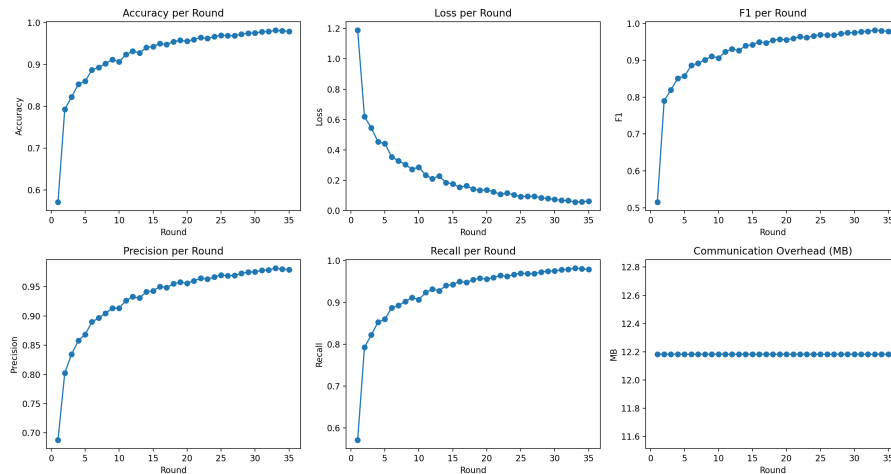


Figure 4.3: Accuracy metrics of FedAvg

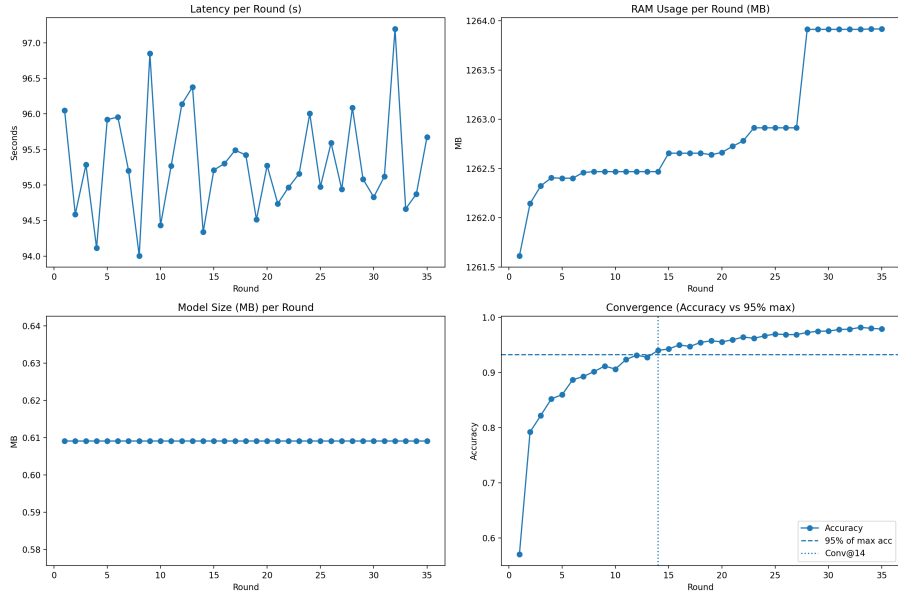


Figure 4.4: Per Round Overheads of FedAvg

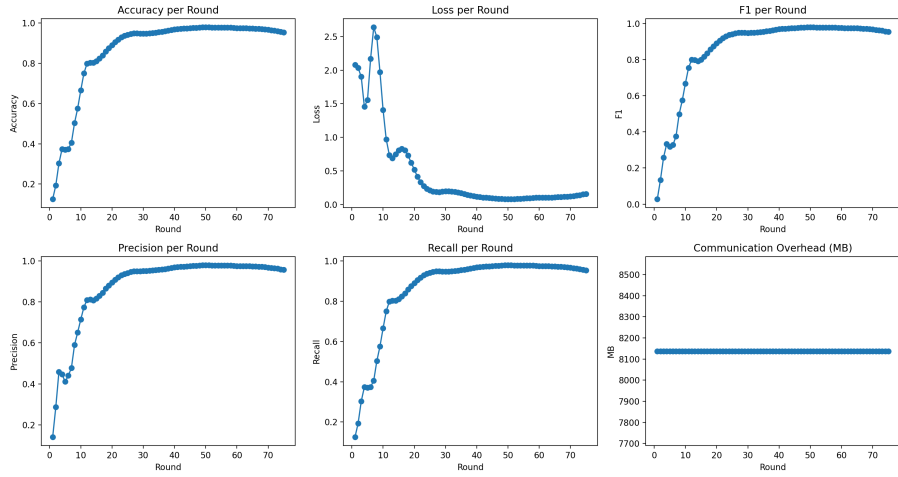


Figure 4.5: Accuracy metrics of FedOpt

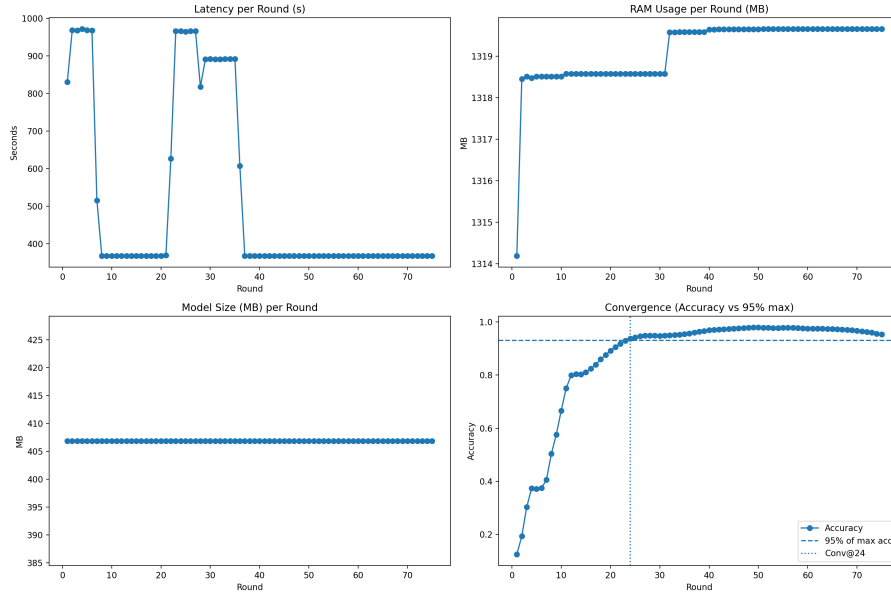


Figure 4.6: Per Round Overheads of FedOpt

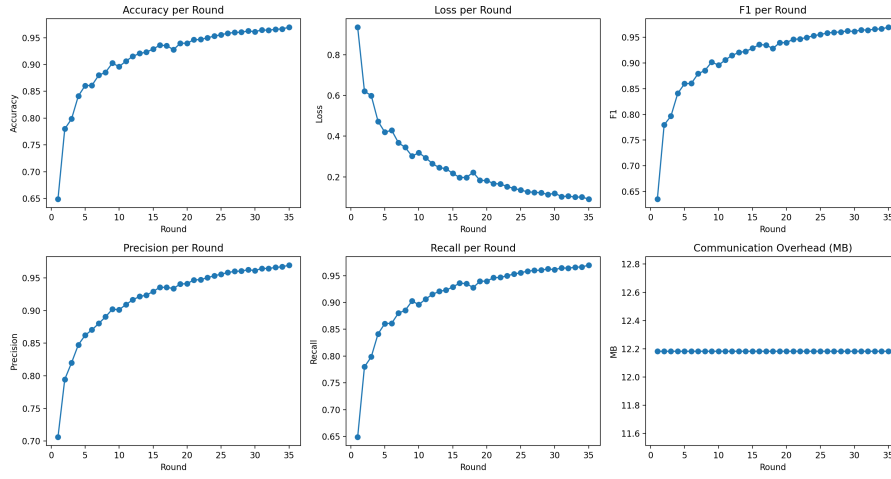


Figure 4.7: Accuracy metrics of FedProx

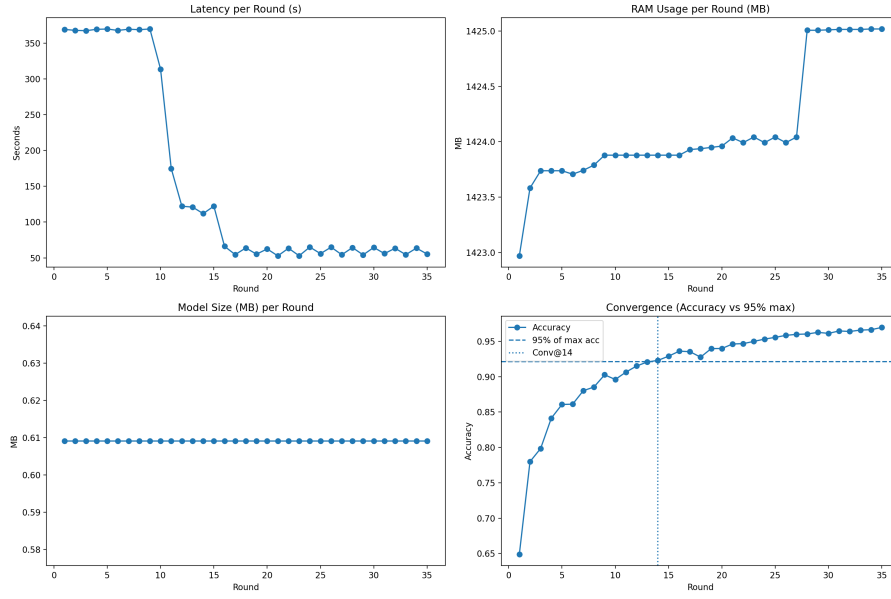


Figure 4.8: Per Round Overheads of FedProx

For CIFAR-10, FedProx achieved 94.95% final accuracy, which is the highest among the three. Meanwhile, FedAvg achieved 94.65% accuracy and FedOpt reached 94.86% accuracy. FedOpt achieved the lowest latency which is 7.58 s/round, followed by FedProx (8.01 s/round) and FedAvg (10.85 s/round). We found model sizes to be identical across all algorithms, which is around 42.7 MB. Memory usage is also very similar for all algorithms, the peak is around 0.09-0.10 MB. Due to the equal model size, communication overhead is also the same (854.03 MB) for all algorithms.

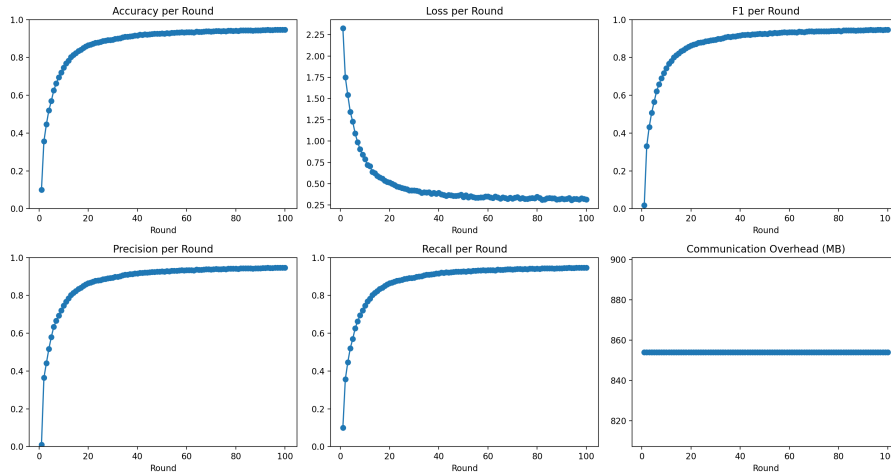


Figure 4.9: Accuracy metrics of FedAvg

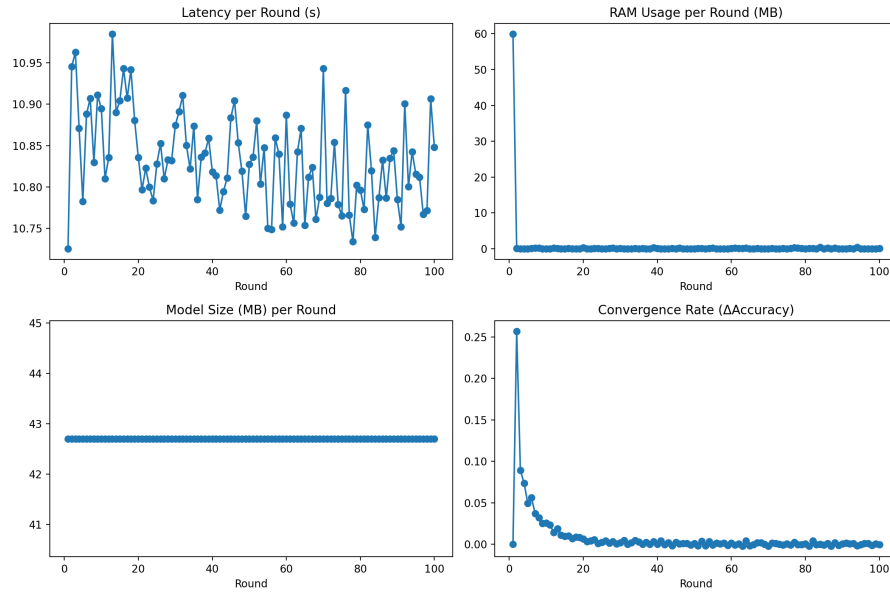


Figure 4.10: Per Round Overhead of FedAvg

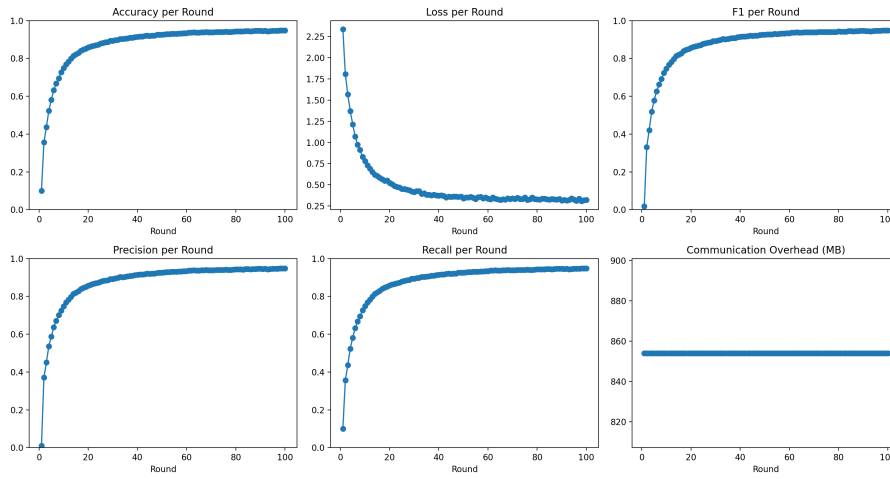


Figure 4.11: Accuracy metrics of FedOpt

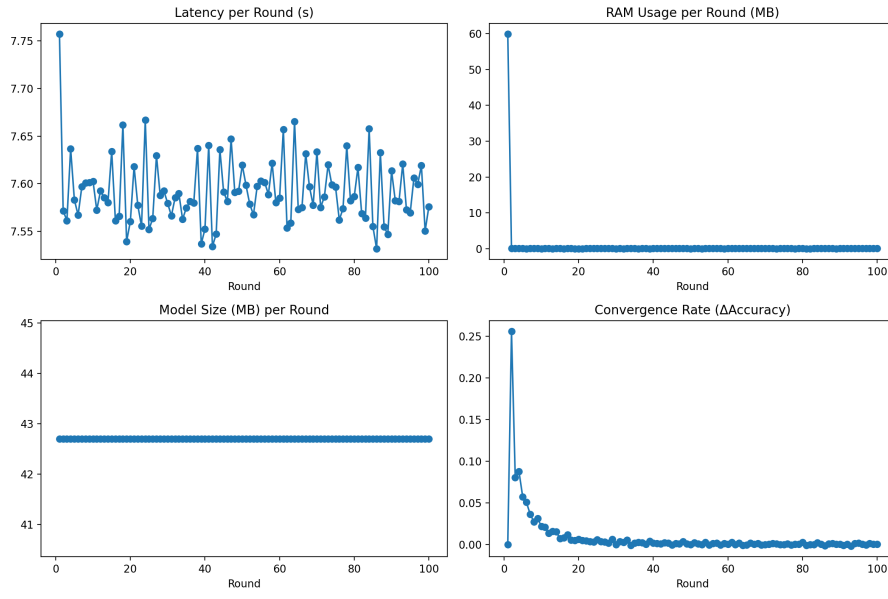


Figure 4.12: Per Round Overheads of FedOpt

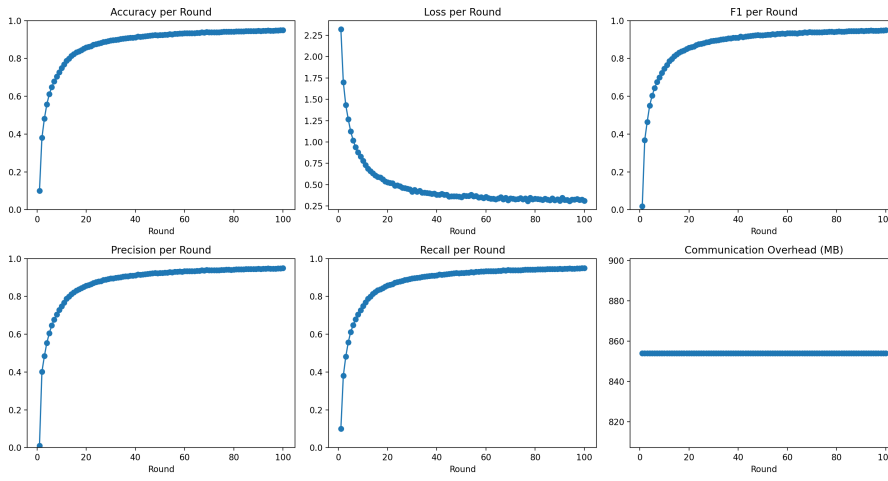


Figure 4.13: Accuracy metrics of FedProx

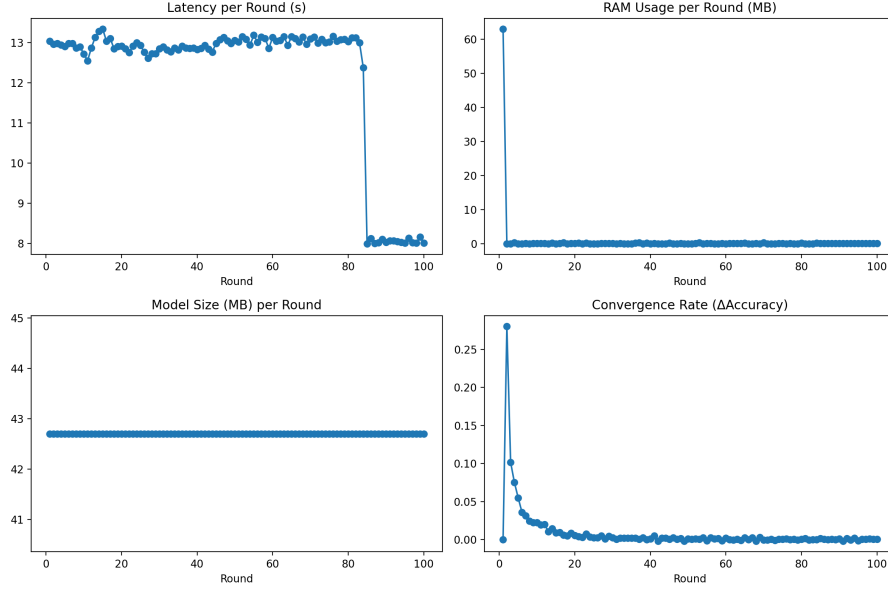


Figure 4.14: Per Round Overheads of FedProx

For MNIST dataset, FedAvg achieved the highest accuracy (87.7%), followed by FedOpt (79.5%) and FedProx (73.4%). During training, FedAvg kept improving early but the convergence rate reduced towards 0 by the end. The convergence rate for FedOpt was steady but slow, where FedProx became stable after 70%. The latency per round of FedAvg was lowest, which is around 13 s/round, followed by FedOpt which is around 28 s/round and FedProx which is around 105-155 s/round. The model size of FedAvg and FedProx were around 0.9 MB and they stayed consistent throughout all rounds. The model size of FedOpt were not logged in these runs. For all methods, memory usage was around 10 MB and communication overheads were around 17.94 MB/round.

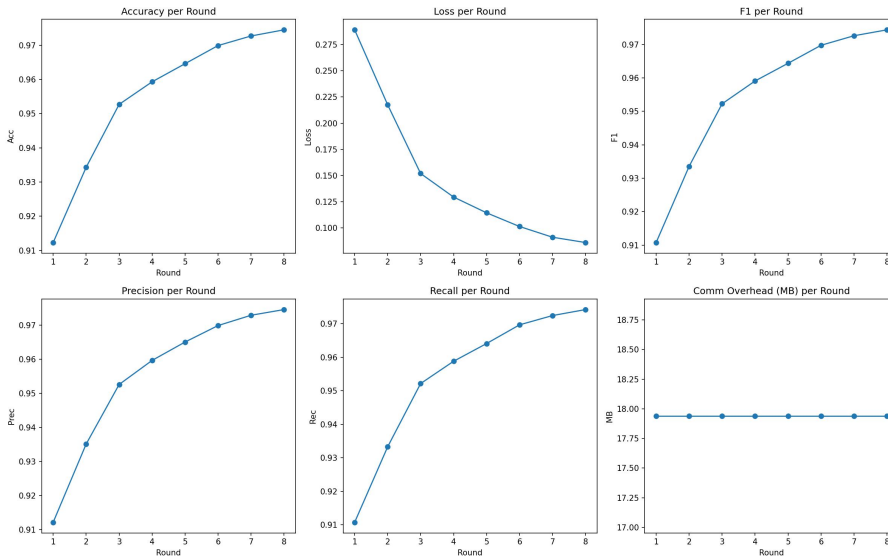


Figure 4.15: Accuracy metrics of FedAvg

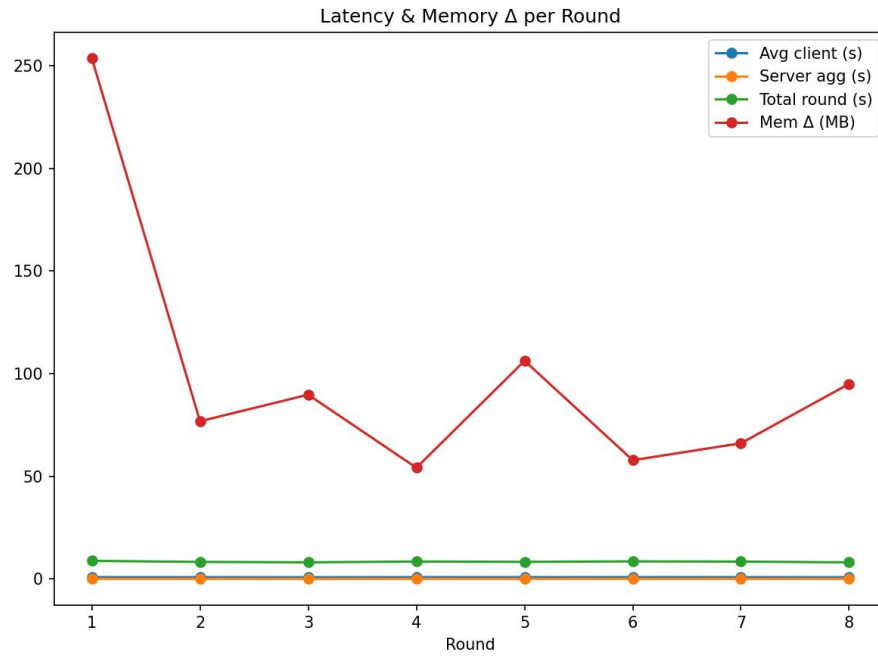


Figure 4.16: Per Round Overheads of FedAvg

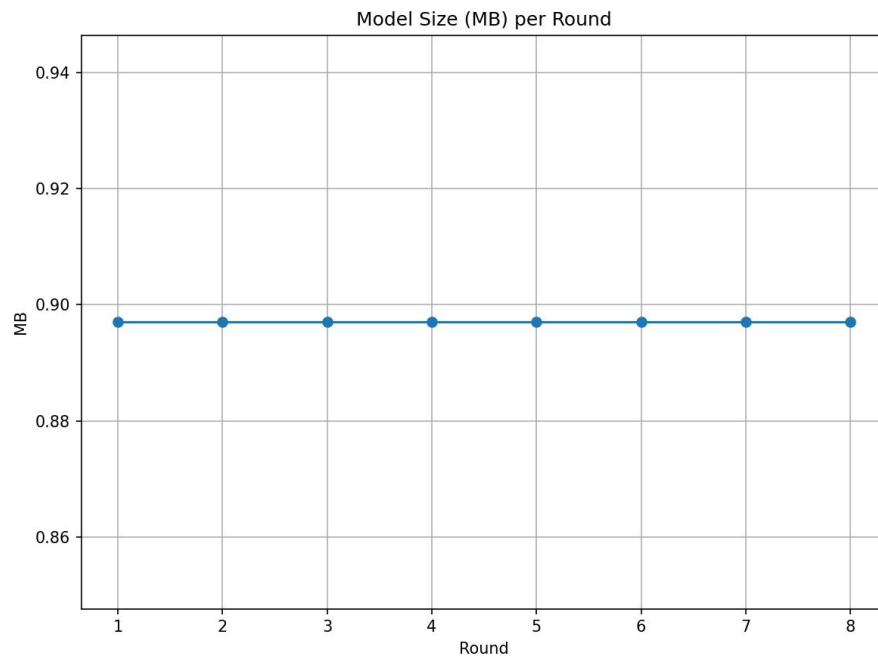


Figure 4.17: Model Size Per Round of FedAvg

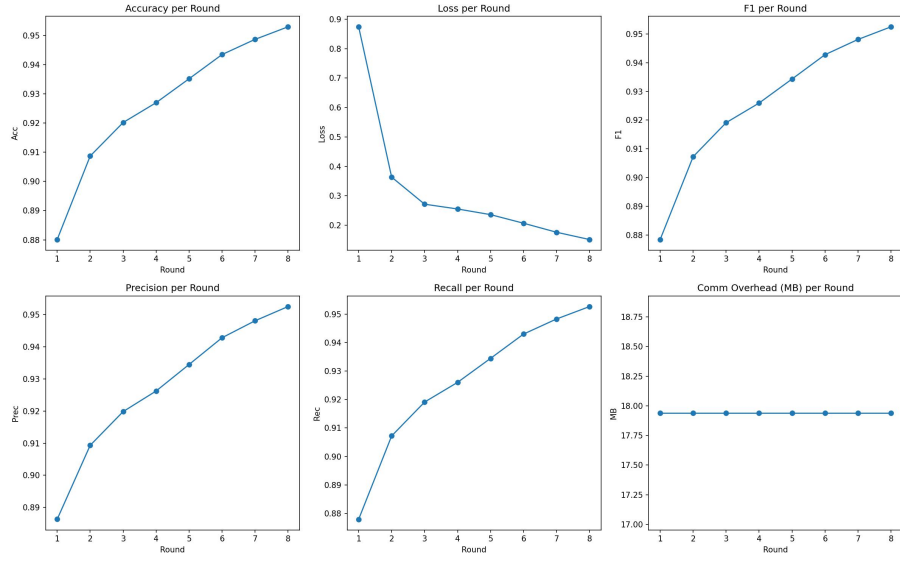


Figure 4.18: Accuracy metrics of FedOpt

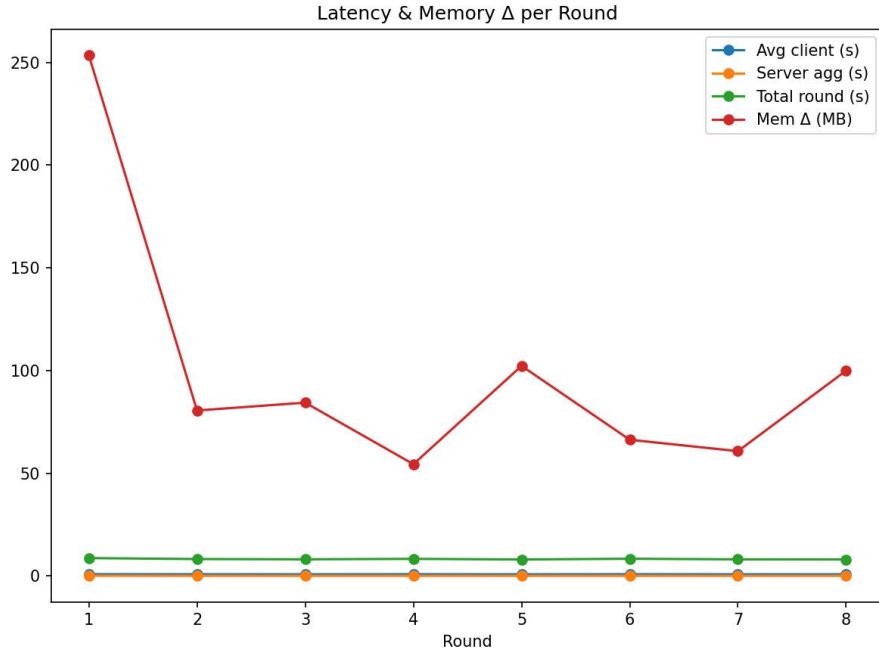


Figure 4.19: Per Round Overheads of FedOpt

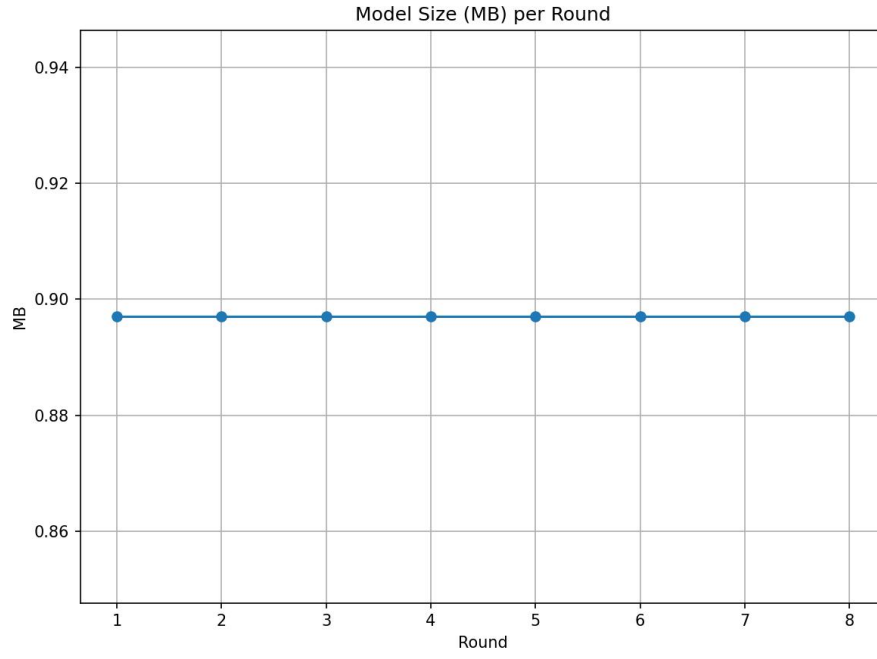


Figure 4.20: Model Size Per Round of FedOpt

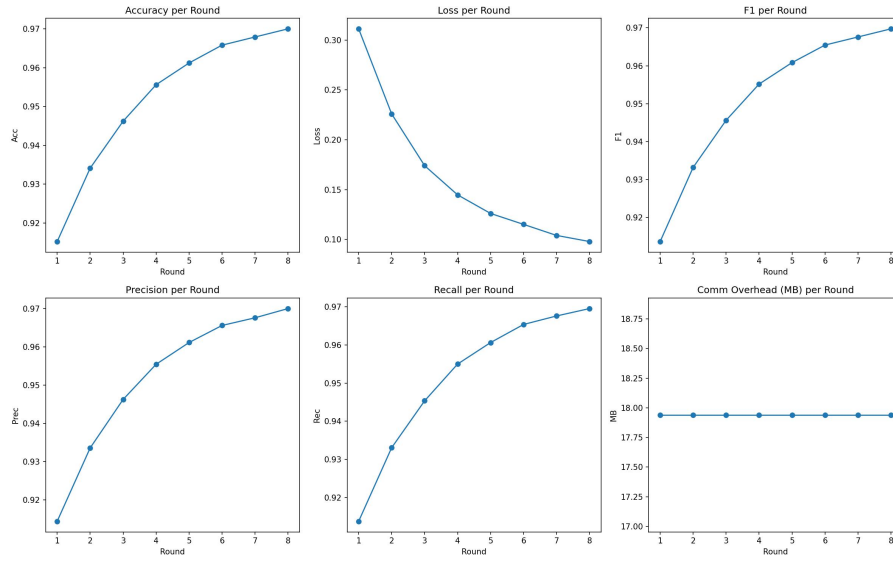


Figure 4.21: Accuracy metrics of FedProx

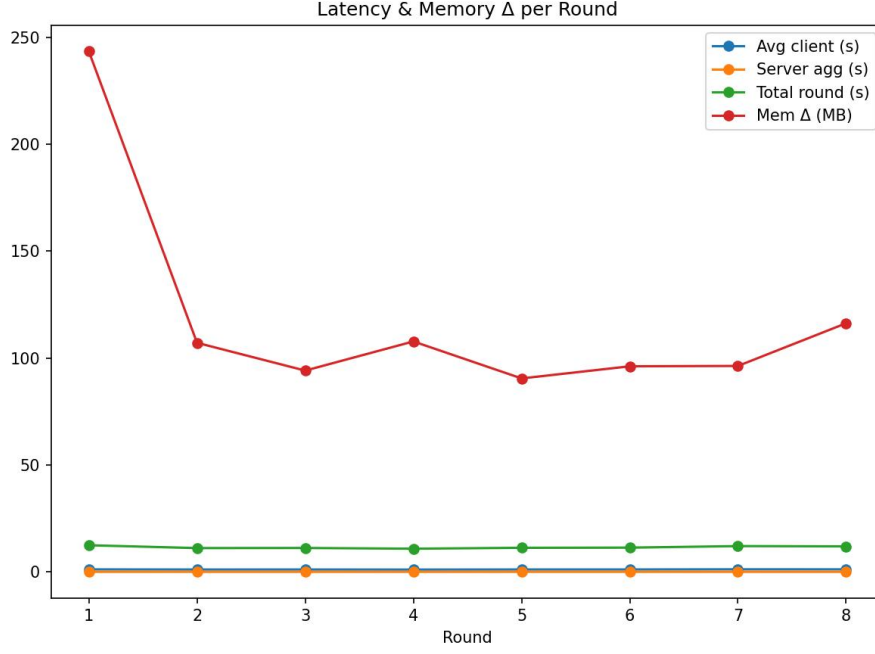


Figure 4.22: Per Round Overheads of FedProx

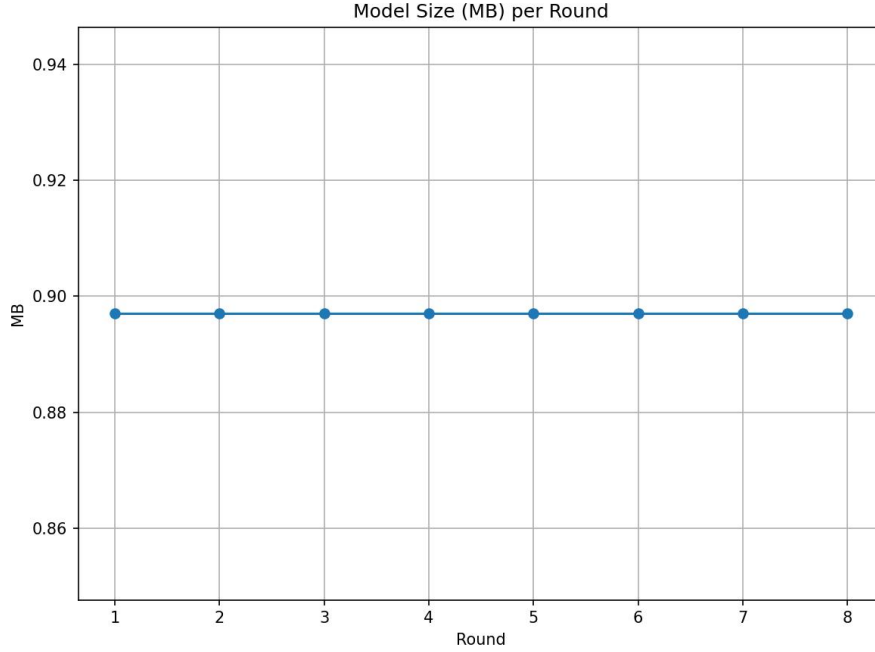


Figure 4.23: Model Size Per Round of FedProx

4.6.4 Challenges and Limitations

For HAR90, to achieve a competitive accuracy, FedOpt required a larger CNN architecture which greatly increased the model size. When we tried to use the slim CNN in FedOpt, it resulted in severe underfitting, indicating the incompatibility between the optimizer and model. We also found that FedProx has slightly slower convergence rate compared to FedAvg. The larger Fedopt model increases storage

and transfer costs, which limits the scenarios we can deploy the model.

For CIFAR-10, we found similar accuracy in all three algorithms, which suggests that dataset and model architecture has limited the maximum achievable accuracy without major design choices. The advantage of FedOpt in convergence speed was less significant compared to other tasks. We got equal model size across all algorithms, which means there is no communication or storage cost advantage for any algorithm.

For MNIST, we found high latency for FedProx due to added proximal term computation and longer server phases. Also, FedOpt was considerably slower than FedAvg even though FedOpt had steady accuracy gains. During training, clients were assigned one class each, which created an extreme non-IID split. This slowed the overall model convergence.

Also, for all the experiments, we conducted the experiments in a simulated environment rather than actual distributed devices. By training on real devices, we might see non-uniform hardware effects.

4.6.5 Result Interpretation

For HAR90 dataset, FedAvg and FedProx proved to be highly effective with compact models, which makes them suitable for low-resource environments. FedOpt can offer quicker convergence, though it requires a larger, more complex CNN for accuracy to match others. We also observed some tradeoffs between the methods. For example, FedOpt offers speed and accuracy at the cost of a larger model size but FedAvg and FedProx offers accuracy and compact model at the cost of speed. The choice of application for these algorithms will largely depend on available bandwidth, storage constraints and latency requirements.

For CIFAR-10, FedProx is slightly better in terms of accuracy and loss. FedOpt has good accuracy with the fastest per round time and FedAvg delivered a well-rounded performance without extra complexity. We can come to the conclusion that FedProx is appropriate for scenarios that require better accuracy. If we need better speed, FedOpt is the best choice. For a good middle-ground, FedAvg is better.

For MNIST, FedAvg proved to be the most practical option when it comes to tight resources, as it provides high accuracy with small model size. FedOpt can be a viable option when optimizer behavior matters more than per-round speed. The low accuracy can be improved after additional tuning. FedProx provides stable learning but has higher latency. Also, the data distribution was extremely skewed due to having only one class per client.

Chapter 5

Conclusion

This study tries to solve the problem of excess data load in cloud environments. It tries to solve the problem through Federated Learning and Data compression. Applied in real-world scenarios this solution can cause paradigm shifts as it delivers a more interactive and faster response from servers taking up less storage space all the while keeping the servers from being overloaded cost-effectively. It would redefine consumer privacy as all data would be stored locally and only the global model would be updated based on the local model training. This would also make it easier on data transmission as compressed data would need fewer packets to send the same amount of information.

The research does have its limitations. Too much compression of the data can distort the information in a way that can make the whole data corrupt. Also, such an amount of compression would negatively affect transmission as losing or misplacing even a slight amount of data in this situation would lead to almost useless data retrieval. FL is applied here mostly on the end devices; so if the end device environment is flawed it can derail the global model and hamper the whole process.

This work demonstrates how integrating data compression with FL can significantly reduce communication overhead, paving the way for more efficient and scalable distributed ML systems.

Bibliography

- [1] M. Mahmudul, A. Mia, S. Biswas, M. Urmi, and A. Siddique, “An algorithm for training multilayer perceptron (mlp) for image reconstruction using neural network without overfitting,” *International Journal of Scientific & Technology Research*, vol. 4, Feb. 2015.
- [2] Y. Su, D. Feng, Y. Hua, and Z. Shi, “Predicting response latency percentiles for cloud object storage systems,” in *2017 46th International Conference on Parallel Processing (ICPP)*, 2017, pp. 241–250. DOI: 10.1109/ICPP.2017.33.
- [3] C. Fang, Y. Guo, N. Wang, and A. Ju, “Highly efficient federated learning with strong privacy preservation in cloud computing,” *Computers & Security*, vol. 96, p. 101889, May 2020. DOI: 10.1016/j.cose.2020.101889. [Online]. Available: <https://doi.org/10.1016/j.cose.2020.101889>.
- [4] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, and D. J. Inman, “1d convolutional neural networks and applications: A survey,” *Mechanical Systems and Signal Processing*, vol. 151, p. 107398, 2021, ISSN: 0888-3270. DOI: <https://doi.org/10.1016/j.ymssp.2020.107398>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0888327020307846>.
- [5] S. Shinde, A. Bozorgchenani, D. Tarchi, and Q. Ni, “On the design of federated learning in latency and energy constrained computation offloading operations in vehicular edge computing systems,” *IEEE Transactions on Vehicular Technology*, vol. 71, no. 2, pp. 2041–2057, Dec. 2021. DOI: 10.1109/tvt.2021.3135332. [Online]. Available: <https://doi.org/10.1109/tvt.2021.3135332>.
- [6] Z. Su, Y. Wang, T. H. Luan, *et al.*, “Secure and efficient federated learning for smart grid with edge-cloud collaboration,” *IEEE Transactions on Industrial Informatics*, vol. 18, no. 2, pp. 1333–1344, Jul. 2021. DOI: 10.1109/tii.2021.3095506. [Online]. Available: <https://doi.org/10.1109/tii.2021.3095506>.
- [7] R. C. Brum, L. Arantes, M. C. Castro, P. Sens, and L. M. A. Drummond, *Evaluating execution times and costs of a federated learning application on different cloud providers*, en, Jul. 2022. [Online]. Available: <https://inria.hal.science/hal-03722055/>.
- [8] D. Durner, V. Leis, and T. Neumann, “Exploiting cloud object storage for high-performance analytics,” *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 2769–2782, Jul. 2023. DOI: 10.14778/3611479.3611486. [Online]. Available: <https://doi.org/10.14778/3611479.3611486>.
- [9] S. Fenanir and F. Semchedine, “Smart intrusion detection in iot edge computing using federated learning,” *Revue D Intelligence Artificielle*, vol. 37, no. 5, pp. 1133–1145, Oct. 2023. DOI: 10.18280/ria.370505. [Online]. Available: <https://doi.org/10.18280/ria.370505>.

- [10] M. Liu, L. Pan, and S. Liu, “Cost optimization for cloud storage from user perspectives: Recent advances, taxonomy, and survey,” *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–37, Feb. 2023. DOI: 10.1145/3582883. [Online]. Available: <https://doi.org/10.1145/3582883>.
- [11] K. Mukherjee, R. Shah, S. K. Saini, *et al.*, “Towards optimizing storage costs on the cloud,” *arXiv (Cornell University)*, Jan. 2023. DOI: 10.48550/arxiv.2305.14818. [Online]. Available: <https://arxiv.org/abs/2305.14818>.
- [12] A. K. Samha, “Strategies for efficient resource management in federated cloud environments supporting infrastructure as a service (iaas),” *Journal of Engineering Research*, vol. 12, no. 2, pp. 101–114, Nov. 2023. DOI: 10.1016/j.jer.2023.10.031. [Online]. Available: <https://doi.org/10.1016/j.jer.2023.10.031>.
- [13] X. Wei, J. Liu, and Y. Wang, “Joint participant selection and learning optimization for federated learning of multiple models in edge cloud,” *Journal of Computer Science and Technology*, vol. 38, no. 4, pp. 754–772, Jul. 2023. DOI: 10.1007/s11390-023-3074-4. [Online]. Available: <https://doi.org/10.1007/s11390-023-3074-4>.
- [14] S. T. Ahmed, V. V. Kumar, T. R. Mahesh, *et al.*, “Fedopt: Federated learning-based heterogeneous resource recommendation and optimization for edge computing,” *Soft Computing*, Jan. 2024. DOI: 10.1007/s00500-023-09542-6. [Online]. Available: <https://doi.org/10.1007/s00500-023-09542-6>.
- [15] Y. Chen, Q. Ke, H. Li, Y. Wu, and Y. Zhang, “X meta: Ssd-hdd-hybrid optimization for metadata maintenance of cloud-scale object storage,” *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 2, pp. 1–20, Mar. 2024. DOI: 10.1145/3652606. [Online]. Available: <https://doi.org/10.1145/3652606>.
- [16] E. A. T. Kamble, “Predictive resource allocation strategies for cloud computing environments using machine learning,” *Deleted Journal*, vol. 19, no. 2, pp. 68–77, Jan. 2024. DOI: 10.52783/jes.692. [Online]. Available: <https://doi.org/10.52783/jes.692>.
- [17] A. Q. Khan, M. Matskin, R. Prodan, C. Bussler, D. Roman, and A. Soyly, “Cloud storage tier optimization through storage object classification,” *Computing*, Apr. 2024. DOI: 10.1007/s00607-024-01281-2. [Online]. Available: <https://doi.org/10.1007/s00607-024-01281-2>.
- [18] M. N, S. H. K. G, S. R, and J. I. R. Nr, “Federated transfer learning for intrusion detection system in industrial iot 4.0,” *Multimedia Tools and Applications*, vol. 83, no. 19, pp. 57 913–57 941, Feb. 2024. DOI: 10.1007/s11042-024-18379-6. [Online]. Available: <https://doi.org/10.1007/s11042-024-18379-6>.
- [19] J. Noor, M. S. I. Sakif, J. J. Mondal, *et al.*, “Sherlock in oss: A novel approach of content-based searching in object storage system,” *IEEE Access*, vol. 12, pp. 69 456–69 474, 2024. DOI: 10.1109/ACCESS.2024.3401074.
- [20] Y. Qi, Y. Feng, X. Wang, H. Li, and J. Tian, *Leveraging federated learning and edge computing for recommendation systems within cloud computing networks*, en, Mar. 2024. [Online]. Available: <https://arxiv.org/abs/2403.03165>.

- [21] S. Syed and E. M. Albalawi, “Optimizing cloud resource allocation with machine learning: A comprehensive approach to efficiency and performance,” *Research Square (Research Square)*, Aug. 2024. DOI: 10.21203/rs.3.rs-4825637/v1. [Online]. Available: <https://doi.org/10.21203/rs.3.rs-4825637/v1>.
- [22] Griffiths, *The latest cloud computing statistics (updated january 2025) — aag it support*, en-GB, Jan. 2025. [Online]. Available: <https://aag-it.com/the-latest-cloud-computing-statistics/>.
- [23] M. Raeiszadeh, A. Ebrahimzadeh, R. H. Glitho, J. Eker, and R. a. F. Mini, “Asynchronous real-time federated learning for anomaly detection in microservice cloud applications,” *IEEE Transactions on Machine Learning in Communications and Networking*, p. 1, Jan. 2025. DOI: 10.1109/tmlcn.2025.3527919. [Online]. Available: <https://doi.org/10.1109/tmlcn.2025.3527919>.
- [24] en. [Online]. Available: <https://ubuntu.com/ceph/what-is-ceph>.
- [25] en-US. [Online]. Available: <https://aws.amazon.com/what-is/iaas/>.